

CHIME：基于 DIMM-PIM 的高效长上下文 Attention-FC 分离式推理研究

CHIME: A Case for Efficient Long-Context Attention-FC Disaggregated Inference with DIMM-PIM

Qingyuan Liu*, Liyan Chen*, Haocheng Wang, Yanning Yang, Dong Du,
Zhigang Mao, Naifeng Jing, Yubin Xia, Haibo Chen
上海交通大学, 中国上海

Abstract—Attention-FC 分离式 (Attention-FC Disaggregated, AFD) 大语言模型推理系统将受内存带宽约束的 Attention 操作卸载到内存资源丰富的加速器 (如 CPU、HBM-PIM), 同时把受计算约束的全连接 (Fully-Connected, FC) 操作保留在 GPU 上。本文首先设计分离式屋顶线模型 (Disaggregated Roofline Model, DRM) 来刻画 AFD 的性能; 该模型揭示, 系统吞吐量受加速器的限制因素约束, 而这一因素可能是内存带宽, 也可能是内存容量。我们观察到, 以往 AFD 系统往往忽视且未能平衡这些约束, 因而导致资源利用不足或吞吐量受限。为此, 我们提出 CHIME: 首个集成 DIMM-PIM 的 AFD 系统; DIMM-PIM 是一类凭借可扩展容量与带宽实现二者平衡的新型加速器。为解决 DIMM-PIM 中分布式协作 DRAM 芯片固有的同步难题, CHIME 采用无气泡流水线和混合粒度重布局来高效计算 Attention。此外, 它通过秩组粒度的通信-计算重叠与对齐预测调度, 最大化跨设备资源利用率。评估结果表明, 与最先进的 HBM-PIM 方案相比, CHIME 可实现最高 $5.15\times$ 的加速。

I. 引言

“无论其他养分多么充足, 植物的生长都受供应量最少的养分所限制。”

—利比希定律 [75], Justus von Liebig (1803–1873)。

在长程推理、复杂代码生成等应用的推动下, 大语言模型 (LLM) 处理和生成更长序列的需求正在快速增长 [16], [21], [29], [65], [81], [82]。在这些长上下文场景中, 解码过程会成为显著的性能瓶颈。此外, 日益增长的 KV cache 对推理系统的内存容量与带宽提出了越来越高的要求。

为应对这一内存挑战, Attention-FC 分离 (AFD) 已成为一种有前景的方案 [27], [28], [30], [61], [71], [72], [83]。AFD 系统对 LLM 推理工作负载进行划分: 将内存密集的 Attention 操作与 KV cache 卸载至内存资源丰富的专用设备 (下文称为“加速器”), 而计算密集的全连接 (FC) 操作仍在 GPU 或 NPU 上执行。例如, 利用基于 HBM 的存内计算 (HBM-PIM), 可以通过显著提高带宽来大幅降低 Attention 延迟 [27], [28], [61]。总体而言, AFD 系统展现出提高吞吐量的潜力。

然而, AFD 架构的这一潜力并不总能直接兑现。我们的研究揭示了一个关键但反直觉的现象: 单纯增强加速器并不总能保证系统性能更好。在使用 OpenR1 [5] 的 GPT-175B 评估中, 我们发现, 即使为 DGX-A100 的 HBM 配置 PIM, 并将带宽从 $1\times$ 提升至 $16\times$, 吞吐量提升仍不足 1%。现有性能模型无法解释这一令人困惑的结果: 例如, 经典屋顶线模型 [76] 孤立地分析设备, 无法捕捉 AFD 系统中分离组件之间的复杂相互作用。

* 共同第一作者。

为解决这一问题, 我们设计了分离式屋顶线模型 (DRM) (第 III 节), 这是首个用于分析 AFD 系统在不同场景下性能的一般模型。DRM 整体刻画加速器和 GPU 的性能, 并特别对二者的相互依赖关系建模。借助 DRM, 我们得到一条指导 AFD 系统设计的原则: 系统吞吐量受最稀缺资源限制, 而这一资源可能是加速器的内存带宽, 也可能是其内存容量。关键在于, 改善任何非瓶颈资源都只会带来递减、甚至可以忽略的收益。这解释了上一段中的现象, 也促使我们设计一种能在容量、带宽和可扩展性之间实现更均衡权衡的新型加速器。

本文表明, 将 PIM 集成到 DIMM 中, 即在主机内存模块内配置 Bank 级处理单元, 是实现这种平衡的一类新型加速器。因此, 我们提出采用 DIMM-PIM 的 AFD 系统。与以往 HBM/GDDR-PIM 方案 [22], [27], [28], [61] 相比, 采用 DIMM-PIM 的 AFD 系统具有三个关键优势。第一, 它在提供远高于标准主机内存带宽的同时, 直接缓解 HBM-PIM 的容量瓶颈, 从而提高总体吞吐量。第二, 它继承标准 DIMM 接口优良的可扩展性和可配置性, 能够适应多样化的内存需求并具有面向未来的能力。第三, 如第 III-C 节所分析, 它具有更高的经济效率: 成本可能低于 HBM/GDDR-PIM, 内存浪费也更少。

然而, 在 AFD 系统中有效利用 DIMM-PIM 并非易事。朴素地集成 DIMM-PIM 会在 PIM 硬件内部和整个推理系统中引入额外同步开销, 包括传输、布局与进度同步, 并可能严重阻塞推理。具体而言:

第一, 与以往 HBM-PIM/GDDR-PIM 架构 [22], [27], [28], [61] 相比, DIMM-PIM 架构包含多个分布式协作 DRAM 芯片, 从而加剧两类同步开销: (1) 数据同步: Attention 计算分布在多个 DRAM 芯片上, 需要通过跨芯片传输来同步数据, 例如为 “softmax” 累加各芯片的部分输出。(2) 布局同步: 基于 DIMM 的数据存储布局可能与 PIM 计算所需布局不一致; 例如, 单个元素可能条带化分布在多个芯片上, 而 PIM 计算要求每个元素完整驻留于单个芯片。因此, 在执行 Attention 之前必须对齐布局。这些开销会在 PIM 执行中造成大量停顿, 使 DIMM-PIM 无法达到其理论性能。

第二, 在 GPU 与 DIMM-PIM 之间分离推理过程还会引入两类同步开销: (1) 数据同步: PIM 需要从 GPU 获取 Q、K、V 张量, 并将 Attention 输出返回 GPU。这些经由 PCIe 的传输会阻塞 PIM 执行。(2) 进度同步: 当 PIM 和 GPU 上的并行操作完成时间不平衡时, 较早完成的设备必须等待, 从而产生空闲“气泡”。可并行任务的执行延迟还会发生变化, 使工作负载对齐更加困难, 进一步加剧这一问题。

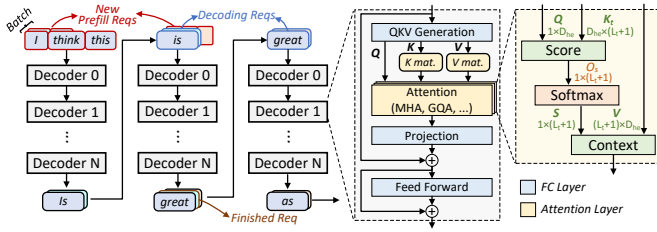


Fig. 1. LLM 推理过程。

为应对上述挑战，我们提出 CHIME，这是首个将新型 DIMM-PIM 设计作为加速器的 AFD 系统，可从硬件内部一直到整个系统降低同步开销。首先，我们设计用于高效 Attention 计算的 DIMM-PIM 硬件 CHIME-PIM（第 V 节）。它分别通过两项关键技术消除数据同步和布局同步开销：无气泡流水线与混合粒度重布局。具体而言，该流水线把数据传输与 Bank PU 的并发执行重叠起来以隐藏传输开销，并在定量分析的基础上采用特定 Head 映射，进一步实现无气泡执行。混合粒度重布局分别在元素级（粗粒度）和比特级（细粒度）执行数据布局变换，以最低延迟解决布局不匹配问题。

其次，我们设计集成 DIMM-PIM 的推理系统 CHIME-sys（第 VI 节），分别通过秩组粒度通信—计算重叠和对齐预测调度降低数据同步与进度同步开销。秩组粒度通信—计算重叠利用能够独立通信与计算的最细粒度，即秩组（rankset）；一个秩组由每个通道中的一个 Rank 组成。该设计使不同秩组能够并行通信和计算，以支持异步数据传输，从而用计算有效隐藏 GPU 与 CHIME-PIM 之间的通信开销。对齐预测调度能够对两个设备上的操作性能建模，并据此选择请求组成子批次，使这些子批次在两个设备上的预测延迟对齐。这样可以以最少的空闲气泡最大化两个设备上的并行执行。

在三个 LLM 模型与三个真实轨迹 [2], [5], [6], [11] 上的综合评估表明，CHIME 通过显著增大批大小，相比最先进的 HBM-PIM 方案 [28], [61] 可实现最高 $5.15\times$ 的加速。

II. ATTENTION-FC 分离式系统的兴起

A. LLM 推理过程

图 1 展示了 LLM 推理过程 [9], [19], [21], [33]，其中包括预填充与解码两个阶段。LLM 模型由一系列层构成，每层包含两类主要操作：(1) Attention 操作 [73]；(2) 全连接（FC）操作，包括 QKV 生成、投影、前馈网络等。MHA 与 GQA 是主流 Attention 算子：MHA 独立计算每个 Head，GQA 则对 Query Head 分组并行处理。Attention 由彼此独立的 Head 完成计算，每个 Head 依次执行 Score ($Q \times K_t$)、Softmax 和 Context ($S \times V$) 计算。

B. Attention-FC 分离式推理系统

Attention 与 FC 操作通常具有不同特征。具体而言，受益于批处理 [8], [41], [78]，FC 操作计算密集，需要计算资源来加速。相反，解码 Attention 通常是带宽密集型操作 [61]，且无法从批处理中获益，因为每个请求都有各自独立且专用的 KV cache。Attention 延迟主要由加载 KV cache 所需的内存带宽决定。考虑到两类操作的不同特征，一些现有工作——即 Attention-FC 分离式推理系统（简称“AFD 系统”）——把 Attention 与 FC 操作分离到不同

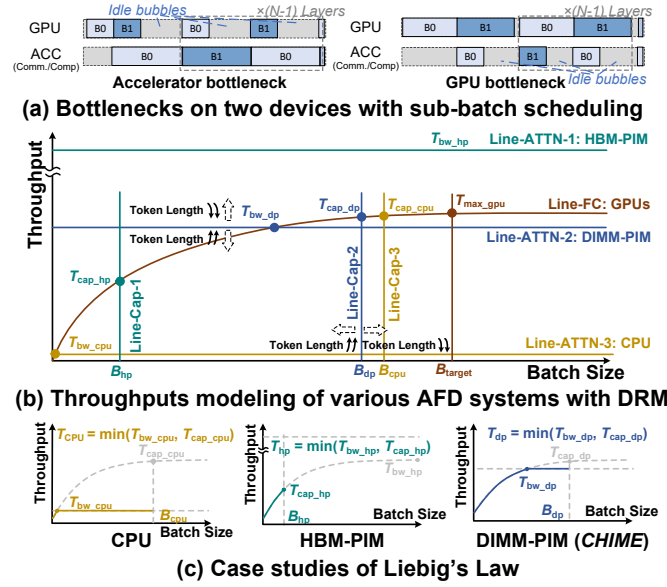


Fig. 2. CHIME 的分离式屋顶线模型与案例研究。“DIMM-PIM”表示 CHIME 的方案，即采用配有 Bank 级 PU 的 DIMM 主机内存作为加速器。“Throughput”表示每秒处理的 Token 数。

硬件平台上。具体而言，它们在计算资源丰富的 GPU/NPU 上对 FC 操作进行批处理，同时把 KV cache 和解码 Attention 卸载到更适合其内存密集特性的平台（称为加速器）。

为提高总体利用率，现有 AFD 系统通常采用子批次调度 [28], [30], [83]，如图 2-a 所示，在两个设备上并行执行两个（甚至更多）交错子批次。推理时，单个批次的执行无法在两个设备上并行；例如，GPU 必须等待 Attention 完成后才能开始 FC。子批次调度允许不同批次的 Attention 和 FC 在两个设备上并发运行，从而能够提高 AFD 系统的吞吐量与资源利用率。

C. 存内计算（PIM）

PIM 通过在 HBM [26], [38], [39], [46], [49]、GDDR [36], [42], [43], [47], [48] 和 DIMM [18], [34], [35], [44] 等内存设备中集成处理单元（PU），提供较高的聚合带宽，是一种有前景的方案。例如，当一个通道中包含两个 DIMM（如每个 DIMM 含两个 Rank、每个 Rank 含 16 个 Bank）时，在 Rank 附近部署 PU（Rank PU）可获得约为主机 CPU $4\times$ 的带宽，而在 Bank 附近集成 PU（Bank PU）可将带宽提高 $30\times$ 以上。因此，一些最先进的 AFD 系统构建在 HBM-PIM [27], [28], [61] 或 GDDR-PIM [22] 等 PIM 设备之上，相比 GPU 可获得显著的性能提升。

III. 使用 DRM 刻画 AFD 系统

本节首先介绍分离式屋顶线模型（DRM），这是一个专为 AFD 系统设计的分析模型，可刻画不同场景下的性能。基于该模型，我们分析现有 AFD 系统的内存瓶颈；这些系统把 Attention 卸载到不同的内存增强设备上，包括提高带宽的 HBM-PIM，以及扩展容量的 CPU（配合主机内存）。根据分析，我们总结“利比希定律”在 AFD 系统中的具体表现，以此指导加速器通过有效增强内存来实现高吞吐量。

A. DRM 建模与瓶颈分析

刻画 AFD 系统吞吐量需要综合考虑两个因素：GPU 侧执行 FC 操作的吞吐量，以及加速器侧执行 Attention 操作的吞吐量。我们首先指出，当两个设备的吞吐量不相等时，推理系统的吞吐量只受成为瓶颈的那个设备影响。例如，在图 2-a 右侧，GPU 是瓶颈，加速器因而出现空闲气泡。此时，提高加速器侧吞吐量只会产生更多空闲气泡，无法提升总体吞吐量。图 2-a 左侧展示了相反的例子。由此得到：

推论 I: AFD 系统的总体推理吞吐量由分别执行 FC 与 Attention 的两个设备中吞吐量较低者决定。

DRM 的构建。图 2-b 给出 DRM 的概念示意图；它对批大小与 Token 吞吐量 (Token/s) 之间的关系建模，从而统一刻画两个分离设备的性能。Token 吞吐量由两个关键度量推导而来：(1) 每秒浮点运算数，它由传统屋顶线模型中依赖批大小的算术强度决定；(2) 浮点运算量，它随批大小增长。Line-FC 表示 GPU 执行 FC 时的 Token 吞吐量；随着批大小增大，该吞吐量持续上升，直至 GPU 得到充分利用，即当批大小达到 B_{target} 时，GPU 达到峰值吞吐量 $T_{\text{max_gpu}}$ 。Line-ATTN-1、2、3 分别表示不同加速器执行 Attention 时的 Token 吞吐量。由于 Attention 的算术强度保持恒定且较低，这些吞吐量通常与加速器内存带宽正相关，而与批大小无关。此外，图 2-b 中的性能曲线既可以来自理论分析，也可以通过性能剖析获得。

带宽约束。根据推论 I，AFD 系统吞吐量由 GPU 和加速器两条吞吐量曲线中的较低值决定。这里可能出现两种情况。第一，GPU 与加速器的吞吐量曲线相交，例如加速器为 CPU 时。当批大小很小时，GPU 吞吐量起初可能低于 CPU，此时 AFD 系统吞吐量由 Line-FC 决定。随着批大小增加，GPU 吞吐量迅速上升到 $T_{\text{bw_cpu}}$ ；此后 AFD 系统吞吐量由受 CPU 内存带宽约束的 Line-ATTN-3 决定。第二，如果 GPU 与加速器的吞吐量曲线不相交，AFD 系统吞吐量始终由较低的曲线决定。例如，当图 2-b 中的加速器为 HBM-PIM 时，吞吐量始终受 GPU 侧 (Line-FC) 限制。

推论 II: 当 Attention 成为瓶颈时，加速器带宽会限制 AFD 系统吞吐量。

容量约束。另一个因素是加速器侧用于存储 KV cache 的内存容量。内存容量不足会限制可容纳请求的最大数量，使 GPU 侧无法充分利用 GPU 资源。在图 2-b 中，Line-Cap-1、2、3 表示不同加速器可容纳的最大批大小，即 HBM-PIM、DIMM-PIM 与 CPU 分别对应的 B_{hp} 、 B_{dp} 、 B_{cpu} 。因此，GPU 执行 FC 的吞吐量也受这些最大批大小限制。例如，当加速器为 HBM-PIM 时，由于该 AFD 系统的批大小至多为 B_{hp} ，Line-FC 上的最大 GPU 吞吐量被限制在 $T_{\text{cap_hp}}$ 以下。当瓶颈位于 GPU 侧时——例如采用 HBM-PIM 作为加速器——AFD 系统吞吐量就受有限批大小约束。

推论 III: 当 FC 成为瓶颈时，用于存储 KV cache 的容量会限制 AFD 系统总体吞吐量。

动态工作负载与屋顶线移动。DRM 是面向设计阶段架构分析的解析模型，其目的并非预测某个给定请求的精确延迟 [32], [70]。不过，DRM 也能说明性能屋顶线如何随运行时工作负载特征变化而移动。例如，上下文变长时，同等容量可容纳的最大批大小会降低，在相同带宽下处理更大 KV cache 的吞吐量也会下降；因此，容量线 (图 2-b 中的 Line-Cap-1、2、3) 向左移动，Line-ATTN-1、2、3 向下移动，而 GPU 的 Line-FC 几乎保持不变。由此可以看出，长上下文会同时加剧带宽和容量两方面的内存瓶颈。

刻画通信开销。DRM 的方法还能够刻画 GPU 与 Attention 加速器之间的通信开销，并将其视作 Attention 开销的一部分。在 DRM 中，通信带宽下降 (例如用 PCIe 3.0 代替 PCIe 4.0) 会使 Attention 性能曲线向下移动。根据第 VI-A 节的详细分析，AFD 系统中的通信开销是恒定的，并由我们的设计加以优化。

AFD 系统的“利比希定律”。综合推论 II 和 III，对于一个执行 FC 的给定 GPU 配置，当 Attention 或 FC 成为瓶颈时，加速器的内存带宽或容量会分别限制吞吐量。进一步结合推论 I，这意味着总体吞吐量由内存容量和带宽二者中吞吐量上限较低的一方决定，我们称之为内存的薄弱点。由此可得，加速器内存对总体吞吐量的影响遵循利比希定律：

利比希定律: AFD 系统吞吐量受内存容量或带宽中较薄弱的一方约束。

尽管利比希定律是适用于任何系统的一般原则，DRM 的分析方法首次揭示了该定律在 AFD 系统中的具体表现。

案例研究。图 2-c 从图 2-b 中提取具体示例作为案例。“CPU”的薄弱点是主机内存带宽有限；因此，即使内存容量足以容纳更大的批次，CPU AFD 系统的吞吐量仍受 $T_{\text{bw_cpu}}$ 约束。“HBM-PIM”的薄弱点是 HBM 容量有限；因此，即使它能够极快地执行 Attention，吞吐量仍受 $T_{\text{cap_hp}}$ 约束。

利比希定律的定量分析。为了定量说明利比希定律，我们把图 2 的示意图扩展为图 3。图 3 对吞吐量上限进行了量化：假设使用 DGX-A100 的 $8 \times A100$ 计算 FC，同时采用不同内存配置的加速器计算 Attention。该图清楚展示了利比希定律，即只有同时扩展容量与带宽才能获得更高吞吐量。若仅扩展其中一项，例如固定容量而扩展带宽，则吞吐量只会随带宽增加而提升到某个临界点。超过该点后，由于容量成为瓶颈，更高带宽不再带来进一步提升。

B. 以往工作的局限

现有工作为增强内存、提高推理吞吐量付出了大量努力。然而，它们主要聚焦于内存瓶颈的一个维度，即容量或带宽。由于忽视利比希定律，其性能收益可能效率不高或并不充分。为说明这一点，我们在图 3 中绘制了不同硬件类型在扩展场景下的配置，包括 HBM2e、GDDR、两种 DDR4 内存及其 PIM 变体。图中列出的部分系统 [22] 并非专为 AFD 设计；我们主要考察其用于执行 Attention 的硬件。可以看出，以往工作在以下方面受到“利比希定律”制约：

内存增强不足。第一，在扩展不足时，这些内存增强技术可能仍无法阻止内存成为吞吐量瓶颈。结合图 3 考察各

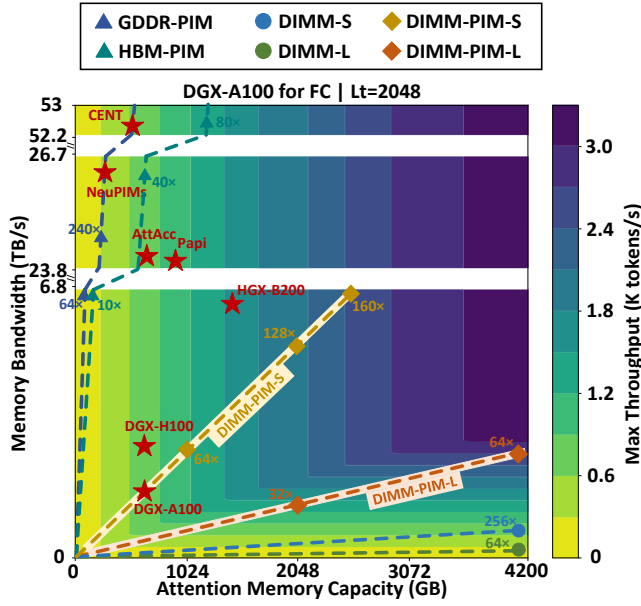


Fig. 3. 存在内存瓶颈时的吞吐量等高线图。上方“ $L_t=2048$ ”表示每个请求的上下文长度为 2048 个 Token。“PAPI”“NeuPIMs”“AttAcc”和“CENT”的内存配置取自相应论文 [22], [27], [28], [61]。“DGX-A100”“DGX-H100”和“HGX-B200” [1], [57], [58] 表示使用 GPU 计算 Attention。DIMM-S 和 DIMM-L 表示两种不同类型的 DDR4 内存。每个点表示相应数量内存芯片下的总容量与总带宽；例如，“DIMM-L”曲线（绿线）上的“64x”点表示 64 个 DIMM-L 内存芯片对应的容量与带宽。我们假设每种内存设备的 PIM 性能均可达到理想情况，即等效内存带宽提高 16x。性能使用 AttAcc [61] 模拟器和 GPT-175B 模型进行模拟。

论文所声称的硬件参数即可说明这一点。具体而言，基于 HBM-PIM 的工作可能受到内存容量不足的制约 [27], [28], [61]，另一些工作则受 DIMM 内存带宽不足限制 [30], [52]。内存增强效率不高。此外，现有工作还存在内存资源过量的问题。例如，尽管当前基于 GDDR/HBM-PIM 的方法 [22], [27], [28], [61] 受容量限制，其带宽却可能比实际需求高出若干数量级，对吞吐量几乎没有帮助。而且，即使对它们进行扩展在理论上能够实现高吞吐量，这种扩展也可能同时造成大量带宽浪费，如图 3 所示。

C. 选择 DIMM 集成 PIM 的方案

基于上述分析，我们发现，将 PIM 与 DIMM 集成（即 DIMM-PIM）是一种有前景的 Attention 加速器。具体而言，DIMM-PIM 通过 Bank 级 PIM 单元增强基于 DIMM 的主机内存，结合了 PIM 与 DIMM 的优势。因此，我们提出集成 DIMM-PIM 的 AFD 系统，其优势如下。

通过更均衡的内存配置获得更好性能。通常，DIMM-PIM 能够缓解 HBM-PIM 的容量瓶颈以及 CPU 卸载的带宽瓶颈。如图 2-b、c 所示，这意味着 DIMM-PIM 有望以更均衡的内存配置提高总体吞吐量。如图 3 所示，DGX-A100 配备 2TB DIMM 内存 (DDR4) 和 640GB HBM，因此为 DIMM 配置 PIM 已明显优于为 HBM 配置 PIM。

可扩展且可配置。DIMM-PIM 通过标准 DIMM 形态提供可扩展性，能够在兼容平台上以即插即用方式扩展容量和带宽，还可通过 CXL 等互连进一步扩展。这使 DIMM-PIM 不仅能够获得更好性能，也可能适应更广泛的场景。

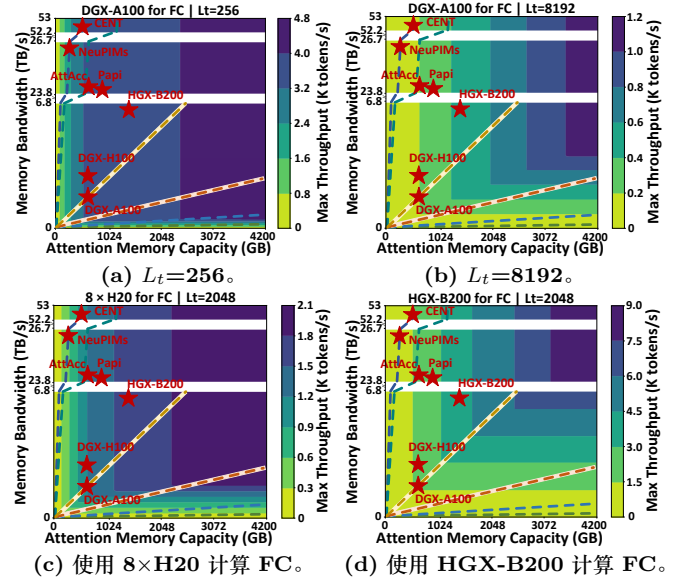


Fig. 4. 四种场景下的吞吐量等高线图。本图与图 3 共用图例。相比图 3 的场景，(a) 和 (b) 改变上下文长度，(c) 和 (d) 则调整用于 FC 计算的 GPU。

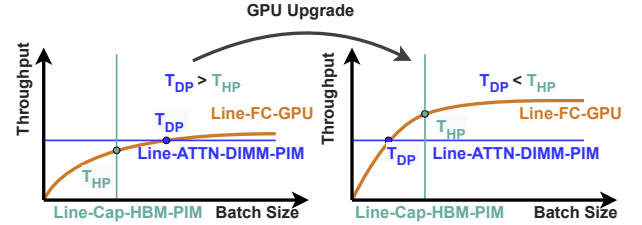


Fig. 5. GPU 升级时的 DRM。 T_{DP} 和 T_{HP} 分别表示受内存带宽约束的 DIMM-PIM 吞吐量，以及受内存容量约束的 HBM-PIM 吞吐量。

经济效率。除性能外，DIMM-PIM 相比 GDDR 和 HBM 方案还具有显著成本优势。第一，HBM2e 的每 GB 成本可能比 DDR4 高 6x 以上 [3], [4], [7]；无论是否集成 PIM，这都是显著差异。第二，图 3 表明，DIMM-PIM 更均衡的性能能够避免 HBM/GDDR-PIM 方案固有的大量资源过度配置，从而以更高的成本效率扩展性能。

D. 讨论：不同场景下的瓶颈迁移

在实践中，AFD 系统可能采用不同的部署配置，从而使内存瓶颈发生迁移。这些部署配置包括 GPU 型号与数量、上下文长度、LLM 参数规模与架构等。基于 DRM 的方法有一项重要贡献：对于给定部署配置，它可以分析每种加速器的内存配置如何影响总体吞吐量。本节进一步讨论不同部署配置可能如何影响内存瓶颈。

不同上下文长度。图 3 与图 4-a、b 展示了不同上下文长度对性能的影响。第一，我们观察到，在不同上下文长度下，DIMM-PIM 更均衡的配置始终具有性能与效率优势，能够在提高吞吐量的同时减少资源过度配置。原因在于，更长的上下文会同时加剧内存容量与带宽瓶颈。第二，我们观察到，在上下文较短时，各基线的性能更为接近，因为内存瓶颈缓解后，增加资源的边际收益会递减。我们在

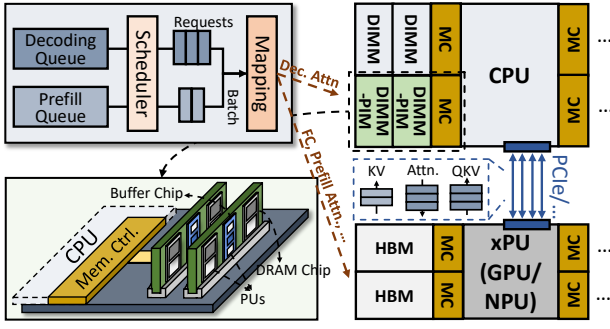


Fig. 6. CHIME 概览。

第 VII-B 节验证了这一分析。上下文长度的影响也可结合图 2-b 进行分析。

其他配置的影响。DRM 方法还可以分析其他配置变化对加速器选择的影响。例如，如图 5 所示，当采用更先进的 GPU 执行 FC 时，DRM 表明 FC 吞吐量曲线上移，从而提高 HBM-PIM 吞吐量而不改变 DIMM-PIM 吞吐量；这会缩小 HBM-PIM 相对 DIMM-PIM 的性能差距，甚至使前者取得优势。图 3 和图 4-c、d 给出了示例：使用 A100 时， $32 \times$ DIMM-PIM-L 的性能优于 AttAcc [61] (图 3)，但与 B200 配合时则低于 AttAcc (图 4-d)。其他场景，例如改变用于 FC 的 GPU 数量，也可沿用类似方法进行分析。

IV. CHIME 概览

A. 设计概览

为了支持集成 DIMM-PIM 的方案，如图 6 所示，我们提出由两部分组成的 CHIME：(1) CHIME-PIM，即我们为高效执行 Attention 专门设计的 DIMM-PIM 硬件；以及 (2) CHIME-sys，即一个可扩展的 AFD 系统，它协调 GPU 与 CHIME-PIM 上的计算，目标是最大化推理吞吐量。

CHIME 的推理工作流。 CHIME-sys 沿用于批次调度技术 [28], [30]，在每轮迭代中选择请求组成子批次。起初，每个子批次中所有请求的 QKV 生成会被批处理并在 GPU 上执行。随后，预填充 Attention 与解码 Attention 分别在 GPU 和 CHIME-PIM 上并发处理。CHIME 汇聚批次内所有请求的 Attention 输出，再批量执行后续操作（投影、前馈等）。GPU 与 CHIME-PIM 通过 PCIe 互连。

解决 DIMM-PIM 集成挑战。如第 I 节所述，集成 DIMM-PIM 会带来架构分离以及随之而来的同步开销：在 DIMM 内部，分布式芯片会同时产生数据同步与布局同步代价；DIMM-PIM 与 GPU 的分离则增加跨设备数据同步和进度同步开销。为解决前一类问题，CHIME-PIM 采用无气泡流水线（第 V-A 节）和混合粒度重布局（第 V-B 节）；为解决后一类问题，CHIME-sys 采用秩组粒度通信-计算重叠（第 VI-A 节）和对齐预测调度（第 VI-B 节）。

V. CHIME-PIM 上的 ATTENTION 加速

本节介绍 CHIME-PIM，这是一种面向高效 Attention 计算的新型 DIMM-PIM 硬件设计。

硬件组件。如图 7-a 所示，CHIME-PIM 在 Bank 和 Rank 两个层级集成相互协作的处理单元 (PU)。Bank PU 采用

DRAM 工艺集成在 DRAM Bank 附近，从 Bank 中读取 KV cache 并执行 Score 与 Context 计算；共享缓冲区则用于向所有 Bank PU 广播输入向量。所有 DRAM 芯片中的 Bank PU 并发执行乘加 (MAC) 操作，并将输出写入结果缓冲区。Softmax 单元、加法器单元和重布局单元采用逻辑工艺集成在缓冲芯片上，作为 Rank PU 的一部分，能够与 Bank PU 异步执行。

CHIME-PIM 的工作流无需修改主机 CPU 内存控制器，具体如下。首先，CPU 通过普通写操作把 PIM 请求卸载到 Rank PU，后者进一步将其解码为 PIM 命令。随后，专用 PIM 控制器经标准 DDR 接口向 DRAM 芯片发出 PIM 命令，以触发相应操作。PIM 命令包括用于元数据设置和数据移动的命令，其中若干关键命令如下。PIM_WR_R 将必要信息写入寄存器，例如计算范式（加法树或累加器）的配置、缓冲区自增索引机制等。PIM_LD_SB 从特定 Bank 把数据加载到共享缓冲区，PIM_WR_SB 则把 Rank PU 中的数据写入共享缓冲区。PIM_MAC 在从 DRAM Cell 和共享缓冲区读取所需数据的同时，在所有 Bank 中执行 MAC 操作。PIM_RD_RB 将所有 Bank 的结果缓冲区数据加载到 Rank PU。

A. 无气泡流水线

为消除 Attention 计算中的数据同步开销，关键在于利用 DIMM-PIM 的独特架构，设计一种用计算隐藏通信的流水化 Attention 执行方式。此外，我们通过定量分析和定制的 Head 映射方法，确保通信成本被完全隐藏。

使用解耦内存总线与算子融合编排流水线。基于两项架构观察，我们为 PIM 中的 Attention 算子编排一种积极重叠计算与通信的流水化执行方式。第一，内部内存总线（用于 Bank PU 访问 Bank）与外部内存总线（用于 Rank PU 访问 Bank PU 或共享缓冲区）可以解耦 [13]；如图 7-b 所示，这使 Bank PU 上的计算算子与 Rank PU 发起或接收的数据传输能够同时执行。第二，受 FlashAttention 算子融合技术 [15]（将 Attention 划分为分块 Tile 计算）的启发，我们编排细粒度算子流水线，在限制 Rank PU 上中间 Head 数据占用的同时最大化并发度。

具体而言，在 Score 阶段，每个 Bank PU 计算某个 Token 对应的输出 O_i 。并将其暂存在本地结果缓冲区后，Rank PU 会立即利用原本空闲的外部总线取回该输出。我们的 Attention 算子以分块 Tile 为粒度运行；每个数据块由单个 Head 在一个或多个 Bank PU 上并行产生的数据构成。数据在加法器单元中累加后，Softmax 单元逐块执行 Softmax。这样一来，Score 与 Softmax 计算便可在 Bank PU 与 Rank PU 之间以数据块为单位形成流水线。处理完所有 Token 后，最终的跨块归一化过程生成全局正确的 Softmax 输出 S ，该过程同样以流式、逐块方式执行。因此，最终 S 元素的计算、将 S 写回 DRAM 芯片的通信，以及后续 Context 计算可以组成流水线。

通过定量分析与特定 Head 映射实现无气泡执行。由于 DIMM 由多个相互协作的 DRAM 芯片组成一个 Rank，KV 矩阵 (cache) 及其计算会分布在这些 DRAM 芯片上。这会增加缓冲芯片与 DRAM 芯片之间的数据传输量，本文称之为跨芯片数据传输。即使跨芯片传输与 Bank PU 执行发生重叠，受 Rank PU 带宽限制，其开销仍可能在流水线执行中产生气泡并成为瓶颈。为应对这一问题，我

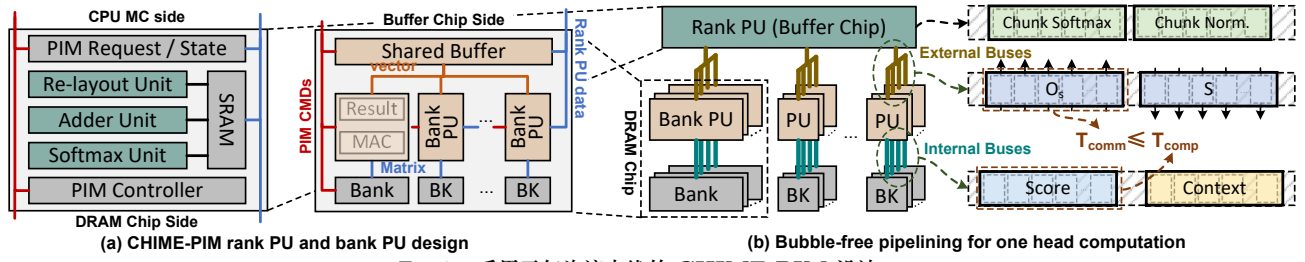


Fig. 7. 采用无气泡流水线的 CHIME-PIM 设计。

们首先结合 LLM 模型配置、硬件参数及 Head 映射方法，对跨芯片数据传输成本进行量化。随后，我们使用特定 Head 映射方法实现无气泡流水线。下述分析以单个 Head 的 Score 计算为例，Context 计算可以采用类似分析。

传输 Score 输出 O_s 的开销记为 T_{comm} ，满足 $T_{comm} = N_{comm}/B_{comm}$ ；其中 N_{comm} 为跨芯片数据传输量， B_{comm} 为相应带宽。具体而言：

$$N_{comm} = L_t \times N_{gqa} \times N_{hc}, \quad B_{comm} = (B_{rk} \times N_{hc})/N_{chips}$$

其中， L_t 为 Token 长度， N_{gqa} 为 GQA 组大小， N_{hc} 为分配给该 Head 的芯片数， B_{rk} 为 Rank 带宽， N_{chips} 为一个 Rank 中的 DRAM 芯片总数。因此，通信时间为：

$$T_{comm} = (L_t \times N_{gqa} \times N_{chips})/B_{rk}$$

这意味着 DIMM 的分布式芯片可能引入额外 $N_{chips} \times$ 的数据传输开销，而 GQA 组大小会进一步加剧该开销。

采用流水化执行后，实现无气泡重叠的关键是 $T_{comm} \leq T_{comp}$ ，即 Bank PU 的计算可以完全覆盖传输。 T_{comp} 可由 $T_{comp} = N_{comp}/B_{comp}$ 计算，其中 N_{comp} 为 Bank PU 读取的数据量 (K Cache)， B_{comp} 为 Bank PU 的聚合带宽。具体而言：

$$N_{comp} = L_t \times E_h \times \lceil N_{gqa}/N_{cmr} \rceil, \quad B_{comp} = B_{bk} \times N_{bk} \times N_{hc}$$

其中， E_h 为 Head 嵌入维度， B_{bk} 为 Bank PU 带宽， N_{bk} 为 Bank 数量， N_{cmr} 表示 Bank PU 中算术单元的计算-内存比。一般而言， $N_{cmr} = 1$ 可最大化 MHA 计算的带宽利用率，而对 GQA-8 计算采用 $N_{cmr} = 8$ [61]。因此，计算时间为：

$$T_{comp} = (L_t \times E_h \times \lceil N_{gqa}/N_{cmr} \rceil)/(B_{bk} \times N_{bk} \times N_{hc})$$

综合考虑 T_{comm} 与 T_{comp} ，确保无气泡的关键条件 $T_{comm} \leq T_{comp}$ 可写为：

$$\frac{L_t \times N_{gqa} \times N_{chips}}{B_{rk}} \leq \frac{L_t \times E_h \times \lceil N_{gqa}/N_{cmr} \rceil}{B_{bk} \times N_{bk} \times N_{hc}}$$

我们发现，在模型与硬件配置给定时，Head 映射方式 (N_{hc}) 是满足该不等式时唯一可调的变量。因此，Head 映射需满足：

$$N_{hc} \leq \frac{E_h \times B_{rk} \times \lceil N_{gqa}/N_{cmr} \rceil}{B_{bk} \times N_{bk} \times N_{gqa} \times N_{chips}} \quad (1)$$

以表 I 和表 II 的配置为例，对于 MHA ($N_{gqa} = 1$)，系统要求 $N_{hc} \leq 8$ ；对于 GQA-8 ($N_{gqa} = 8$)，则要求 $N_{hc} \leq 1$ 。本文对 MHA 采用 $N_{hc} = 8$ ，对 GQA-8 采用 $N_{hc} = 1$ ；这是因为将一个 Head 映射到更多芯片有望简化 Rank 级负载均衡，使模型中的 Head 能够映射到更多 Rank。

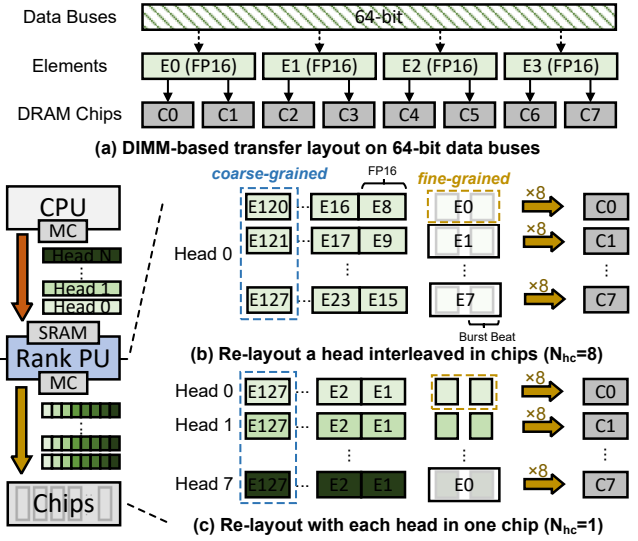


Fig. 8. 以 8×8 芯片为例的混合粒度重布局。E0-E127: 一个 Head 中的元素 0-127; C0-C7: DRAM 芯片 0-7。

B. 混合粒度重布局

除了特定 Head 映射布局之外，包含分布式 DRAM 芯片的 DIMM 还会约束元素布局，如图 8-a 所示。为在数据总线上连续传输，单个元素可能跨越多个 DRAM 芯片 (例如，一个 FP16 元素跨两个 $\times 8$ 芯片)，这从根本上阻止了 PIM 执行。以往由 CPU 辅助的重布局方案 [10], [18], [24], [37], [45] 需要反复访问内存，产生不可忽略的开销 (第 VII-C 节)。为解决这一问题，我们提出混合粒度重布局：利用 Rank PU 的重布局单元，在通信过程中进行在线数据变换。

图 8 左侧以数据卸载为例展示了重布局过程。QKV 向量首先缓存在 Rank PU 的片上 SRAM 中，而不是直接写入 DRAM Bank。完成重布局后，向量才被存入 DRAM 芯片。重布局单元分别通过细粒度与粗粒度重布局来解决元素映射和 Head 映射不匹配问题。第一，在细粒度重布局中，重布局单元确保每个元素完整驻留在单个芯片上。如图 8 所示，单个元素的各比特被排列到连续的突发拍 (burst beat) 中，以存入一个芯片 (棕色虚线框)。例如，元素 0 (E0) 的 16 个比特在两个连续突发拍 (位于同一次 DDR 突发传输中) 内传输，因而存入芯片 0 (C0)。第二，在粗粒度重布局中，重布局单元把每个 Head 映射到 N_{hc} 个芯片。来自一定数量 Head 的元素会一同调度，以遵循 Head 映射 (蓝色虚线框)。例如，在图 8-b 的 $N_{hc} = 8$

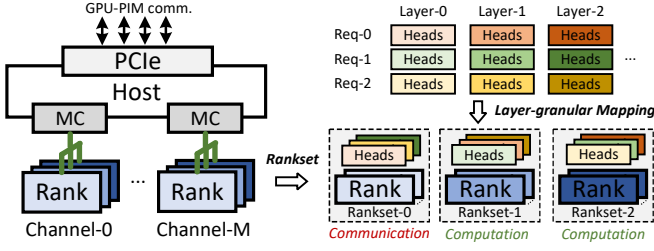


Fig. 9. 以秩组粒度重叠通信与计算，隐藏数据传输开销。

情况下，重布局后每个突发拍包含来自单个 Head 的元素，因此一个 Head 被传输到 8 个芯片。在图 8-c 中，来自 8 个 Head 的元素置于一个突发拍中，因此每个 Head 只传输到单个芯片。数据载回时，重布局执行逆向过程。

VI. 跨设备协同推理

本节介绍 CHIME-sys，即集成 CHIME-PIM、通过软硬件协同设计实现高吞吐量的 AFD 系统；它旨在解决设备间数据同步与进度同步的开销问题。

A. 秩组粒度通信-计算重叠

GPU 与 CHIME-PIM 之间的数据同步包括传输以下数据：(1) 预填充 K、V（与输入长度成正比）；(2) 解码 QKV（每个请求一个 Token）；(3) 解码 Attention 结果（每个请求一个 Token）。理想情况下，采用子批次调度后可以异步传输这些数据，例如在执行一个子批次时发送另一个子批次的解码 QKV。然而，GPU-PIM 数据通信与 CHIME-PIM 上的 Attention 计算共享内存总线，这意味着通信会阻塞计算。这种阻塞妨碍异步数据传输并带来不可忽略的开销，因为 PCIe 带宽 [64] 可能比 CHIME-PIM 的带宽低若干数量级（表 I）。

我们发现，挑战源于粒度过粗，即把所有 Rank 视作整体，只能统一执行通信或计算。解决该问题的关键，是找到可独立、并发执行通信与计算的最细粒度。

秩组粒度通信-计算重叠。我们的关键观察是：由于共享内存总线，在通信期间，一个通道内同一时刻只能访问一个 Rank，而其他 Rank 保持空闲。这促使我们提出秩组 (rankset)：它由每个通道中的一个 Rank 构成，是独立通信与计算的基本粒度。因此，一个秩组是能够充分利用所有通道的最小 Rank 集合。如图 9 所示，每个通道包含三个 Rank，从而形成三个秩组。当一个秩组进行通信时，其他秩组可以独立计算而不被阻塞，因此在通信期间仍可保留 2/3 的计算能力。

此外，我们利用各层 KV cache 大小相同这一特征，实现秩组粒度负载均衡。具体而言，我们以层为粒度、采用交错方式存储每个请求的 KV cache。这样可均衡各秩组的数据传输负载，避免传输数据最多的秩组拖慢总体进度。

B. 对齐预测调度

为避免两个设备间并行操作进行进度同步所产生的空闲气泡，CHIME 调度器的关键在于正确选择请求组成子批次，使两个设备上并行操作的执行延迟对齐。然而，LLM 的自回归特性使这种对齐颇具挑战，因为执行延迟会随已处理 Token 数、批大小等因素变化。以往 AFD 系统未能实现这一目标：对于基于 HBM-PIM 的 AFD 系统 [28]，

[61]，PIM 侧延迟可能始终小于 GPU 侧（如图 2-b 所示）；基于 CPU 的 AFD 系统 [30] 则未在调度时对各操作执行延迟建模。此外，CPU 侧计算时间可能受其他 CPU 应用干扰，因而缺乏可预测性 [17], [53], [54]。

性能建模的机会。为解决这一挑战，CHIME 提出对齐预测调度，其关键特征是对两个设备的执行延迟进行建模与预测，以帮助对齐并行执行延迟。利用这一能力，调度器选择请求组成子批次，使其在两个设备上的预测延迟对齐，如图 10 所示。CHIME 利用以下机会实现性能可预测性。第一，CHIME-PIM 上的执行不受干扰。第二，影响批次性能的因素（例如批大小、已处理 Token 数等）均可预先获得。第三，大量既有工作已经探索 GPU 执行预测方法 [14], [23], [77]。

C. 调度实现细节

模型特征。CHIME 使用以下参数描述子批次 i ： c_{p_i} ，各预填充请求的 Chunk 大小列表； f_{p_i} ，各预填充请求已完成 Token 数列表； f_{d_i} ，各解码请求（在每个 Rank 上）已完成 Token 数列表。两个子批次的延迟可建模如下：

$$T_{GPU_0} = t_p(c_{p_0}, f_{p_0}) + t_{batch}(c_{p_0}, f_{d_1}) \quad (2)$$

$$T_{PIM_0} = t_d(f_{d_0}) + t_{comm}(f_{d_0}, c_{p_1}) \quad (3)$$

$$T_{GPU_1} = t_p(c_{p_1}, f_{p_1}) + t_{batch}(c_{p_1}, f_{d_0}) \quad (4)$$

$$T_{PIM_1} = t_d(f_{d_1}) + t_{comm}(f_{d_1}, c_{p_0}) \quad (5)$$

式 2、4 表示 GPU 侧延迟，其中包含预填充 Attention 与批量 FC 操作延迟。式 3、5 表示 Rank 粒度的 CHIME-PIM 延迟，其中包含解码 Attention 延迟以及与之重叠的数据传输开销。

调度策略。调度策略通过以下步骤选择请求组成子批次，使跨设备并行执行对齐：

- 1) 向每个子批次加入一个预填充请求；如果没有预填充请求，则跳过此步骤。
- 2) 每加入一个预填充请求（意味着 T_{GPU} 增加），CHIME 就向每个子批次加入 N 个解码请求，并以负载均衡方式将其分配给各 Rank。随后，它预测每个子批次的 T_{PIM} 与 T_{GPU} 。若 $T_{PIM} < T_{GPU}$ ，则继续加入 N 个解码请求，直至 $T_{PIM} > T_{GPU}$ 。
- 3) 若仍有预填充请求，则重复步骤 (1)，直至 PIM 内存耗尽。当内存已满且气泡位于 PIM 侧时，对每个子批次最后一个预填充请求进行分块，动态调整 T_{GPU} ，使其尽可能接近另一个子批次的 T_{PIM} 。

N 越大，批次执行时间的调整粒度越粗，但越有利于在不同 Rank 之间实现请求负载均衡。在我们的实现中，MHA 具有更多 Head，且可将这些 Head 均匀分配到各芯片，因此 MHA 的 N 设为 1，而 GQA 的 N 设为 16。

模型选择。下面介绍 CHIME 如何对不同推理操作的执行延迟建模。第一，为建模 T_{GPU} ，我们采用随机森林回归 (RFR)，因为它具有增量学习能力、低延迟和高精度三项优势。大量既有工作已证明 RFR 能够高效预测执行延迟 [53], [80]；其他具有类似特征的模型也可使用。第二，为建模 T_{PIM} ，考虑到 CHIME-PIM 执行具有可预测性（即执行时间与计算或传输的 Token 数呈线性关系），我们可以使用简单快速的线性模型来建模 t_{comm} 和各 Rank 的

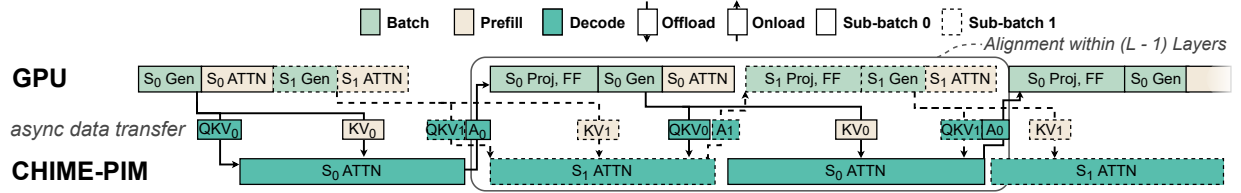


Fig. 10. 使用 CHIME-sys 协调跨设备推理。CHIME 隐藏通信成本，并对齐跨设备并行执行。“ATTN”表示解码或预填充 Attention，“Gen”表示 QKV 生成，“Proj, FF”表示投影与前馈操作。

TABLE I
模拟器细节。

GPU 配置		加速器配置	
处理器	8 A100	CPU	2TB + 406GB/s
容量	640GB	HBM-PIM	640GB + 260.8TB/s (GPU 侧) 320GB + 130.4TB/s (扩展)
带宽	16.3TB/s	DIMM-PIM	2TB + 1.6TB/s (R-PIM) 2TB + 13.0TB/s (CHIME)
DIMM / DIMM-PIM: DDR4-3200			
层级结构	2 Ranks (8 Chips) × 4 Bank Groups × 4 Banks		
DRAM 时序	BL=4:CCD=4:RRD=4/8:RCD=22:RAS=52:RP=22:RC=74: CL=22:WL=16:CDLR=4/12:WR=24:CCDL=8:RTP=12		

* 对于 GQA-n, Bank PU 的计算-内存比 $N_{cmr} = n$ 。

TABLE II
评估所用 LLM 模型。

模型	层数 N_l	Head 数 N_h	嵌入维度 D_e	类型	TP	DP
OPT-66B	64	72	9216	MHA	2	4
QWEN-72B	80	64	8192	GQA-8	4	2
GPT-175B	96	96	12288	MHA	8	1

* 精度: FP16; Head 嵌入维度: $E_h = D_e / N_h = 128$ 。

t_d 。总体 t_d 由执行时间最长的 Rank 决定。模型可由以下公式描述 (以子批次 0 为例):

$$T_{PIM_0} = \text{Linear}(\sum f_{d_0}, \text{len}(f_{d_0}), \sum c_{p_1})$$

$$T_{GPU_0} = \text{RFR}_p(c_{p_0}, f_{p_0}) + \text{RFR}_{\text{batch}}(\sum c_{p_0}, \text{len}(f_{d_0}))$$

运行时性能剖析。CHIME 采用运行时性能剖析，在推理过程中收集延迟及相应批次信息。CHIME 维护的数据集会随采集数据动态演化，并用于增量更新模型，从而适应执行环境的变化。此外，我们按 8:2 划分收集到的 1000 个数据点，构成训练集和测试集，用于评估 CHIME 调度器的预测模型。实验结果表明，该模型具有良好的性能可预测性，预测相对误差低于 $\sim 1\%$ (中位数 $< 0.5\%$)。

VII. 评估

A. 方法

实验设置。按照既有工作 [12], [28], [69]，我们的目标是在长上下文、解码占主导的场景中实现高推理吞吐量。评估基于 DGX-A100 [57] 开展。GPU 侧集成 8 块 NVIDIA A100 GPU，每块配有 5 组总容量 80GB 的 HBM2e，FP16 总计算能力为 156 TFLOPs。各 GPU 通过 NVLink [74] 互连。CPU 侧有 16 个通道和总容量 2TB 的 2 个 DIMM，并配置本文提出的 PIM。我们开发了一个集成以下两部分的模

TABLE III
真实工作负载轨迹。

轨迹	均值 L_{in}	标准差 L_{in}	均值 L_{out}	标准差 L_{out}
OpenR1-Math-220k [5]	96.0	75.1	12684.1	8464.6
Dolphin-r1 [2]	201.9	563.0	3926.2	4216.0
OpenThoughts-114k-math [6]	89.4	66.7	6366.7	4662.9

拟器: (1) AttAcc [61]，用于 GPU 的基于屋顶线模型的模拟; (2) CHIME-PIM-sim，即基于修改版 DRAMSim3 [51] 的 CHIME-PIM 轨迹驱动模拟。我们加入 PIM 命令、相关时序约束和基于 FIFO 的调度方法，以进行周期级模拟。基线系统。我们将 CHIME 与一个 GPU 基线和四个 AFD 基线系统进行比较: (1) 仅 GPU: 完全在 GPU 上执行 LLM 推理; (2) GPU + HBM-PIM: 为所有现有 GPU HBM 配置 Bank 级 PU; (3) GPU + HBM-PIM-EXT: 配置额外的分离式 HBM-PIM; 与“HBM-PIM”基线不同，“HBM-PIM-EXT”中的 HBM-PIM 不存储模型参数; (4) GPU + Rank 级 DIMM-PIM (R-PIM): 为所有 DIMM 配置 Rank 级 PU; (5) GPU + CPU 卸载: 使用 CPU 作为加速器。对所有基线系统的模拟均统一假定在 Attention 计算期间最大化利用加速器侧带宽。表 I 列出硬件规格。GPU、CPU 和 DIMM (-PIM) 配置遵循 DGX-A100 系统的配置。对于 HBM-PIM-EXT，考虑到 HBM2e 的价格是 DDR4 的 $6\times$ 以上 [3], [4], [7]，我们配置 20 个 HBM-PIM 模块 (320GB)，使其容量约为 CHIME 可用容量的 1/6。

LLM 模型。我们使用模型规模各异的三个 LLM 评估 CHIME: OPT-66B、QWEN-72B 和 GPT-175B，数据精度均为 FP16。综合考虑 GPU 内存容量与 GPU 间通信开销，我们采用表 II 所列的不同并行配置。

工作负载。我们使用三个真实 LLM 推理数据集: OpenR1-Math-220K [5]、OpenThoughts-114k-math [6] 和 Dolphin-r1 [2]。这些数据集均用于评估长上下文 LLM 推理性能。轨迹细节见表 III。此外，为评估短上下文场景，我们加入 Dolphin-short 轨迹，将 Dolphin-r1 中每个文本条目缩短至原长度的 1/10。

B. 端到端性能

我们使用不同基线，评估每个 LLM 模型与工作负载组合的端到端吞吐量。每次评估均从轨迹中随机选择并执行 1000 个请求。总体吞吐量通过输出 Token 总数除以总执行时间计算。

真实轨迹上的吞吐量。图 11-a、b、c 给出 CHIME 相对五个基线的归一化吞吐量。结果表明，在不同设置下，CHIME 的吞吐量均为所有系统中最高: 相对 HBM-PIM

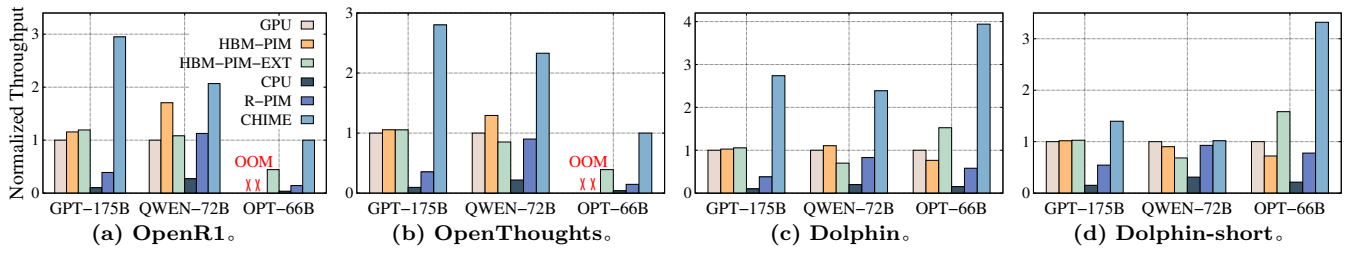


Fig. 11. 真实轨迹上的端到端推理吞吐量。仅 GPU 基线的吞吐量归一化为 1。对 OPT-66B 模型，有两条轨迹发生 GPU 显存不足 (OOM)，因此将 CHIME 的吞吐量归一化为 1。在情形 (d) 中，请求长度缩短为原来的 1/10。

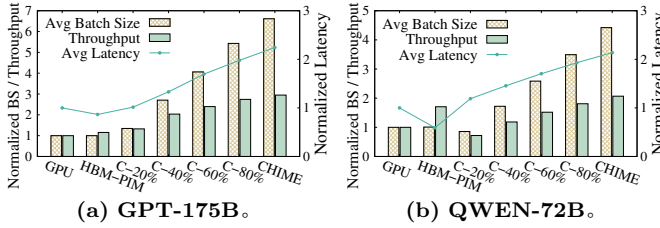


Fig. 12. CHIME 性能提升的分解。结果使用 OpenR1 轨迹评估。“C-N%”表示 CHIME 仅有 N% 的内存容量可用。“BS”表示批大小，“Latency”表示每批次延迟。GPU 基线的结果归一化为 1。

基线最高提升 5.15 $\times$ ，相对 HBM-PIM-EXT 基线最高提升 3.45 $\times$ ，相对仅 GPU 基线最高提升 3.94 $\times$ ，相对 R-PIM 基线最高提升 7.21 $\times$ 。这些改进源于以下原因。第一，相比基于 HBM 的基线，CHIME 能够采用大得多的批大小。例如，对 GPT-175B 而言，CHIME 使用 2TB 主机内存存储 KV cache；部署模型后，GPU 与 HBM-PIM 基线总共仅余约 310GB 内存，而在相同成本预算下，HBM-PIM-EXT 也只有 320GB 内存。第二，CPU 与 R-PIM 基线受带宽限制。例如，通过 Bank 级 PU，CHIME 可提供约为 R-PIM 8 $\times$ 的带宽。

我们观察到，HBM-PIM 的子批次方法可通过增强并行性高效实现高吞吐量，展现出较大潜力；不过，我们的分析也揭示了它在使用 OPT-66B 处理 Dolphin 轨迹时的局限。将一个批次拆分为多个子批次会减小批大小，导致 GPU 严重利用不足，性能甚至低于传统仅 GPU 实现。相反，CHIME 可受益于大得多的批大小，因此实现更高吞吐量。

性能提升来源分解。我们分析了相对 GPU 与 HBM-PIM 基线的性能提升来源。图 12 展示端到端评估得到的吞吐量、平均批大小（两个子批次之和）以及平均每批次延迟。相比 GPU 和 HBM-PIM，CHIME 将批大小提高 6.6 $\times$ ，而每批次延迟仅增加 2.2 $\times$ 。这表明，CHIME 的吞吐量提升主要源自批大小增大以及相应的 GPU 利用率提升。

此外，图 12 展示了批大小增长影响吞吐量的趋势。我们手动限制 CHIME 的可用内存容量并测量相应性能。随着可用内存从 10% 逐步增加到 100%，批大小线性增长。但由于每批次延迟也会增加，吞吐量的提升速率逐渐下降。这一观察与图 2 的分析一致：增大批大小给吞吐量带来的边际收益会递减。

短上下文场景中的性能。如图 11-d 所示，在短上下文场景中，CHIME 仍然通过实现更大的批大小，获得高于 GPU 与 HBM-PIM 基线的吞吐量。然而，相比图 11-c，CHIME

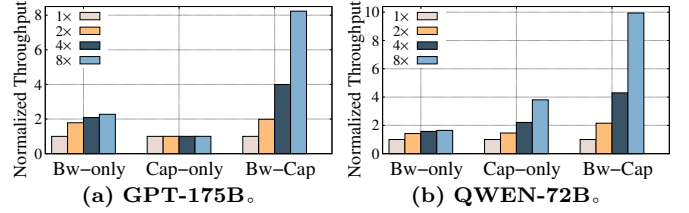


Fig. 13. 可扩展性分析。结果使用 OpenR1 轨迹评估。基础内存配置的性能归一化为 1。

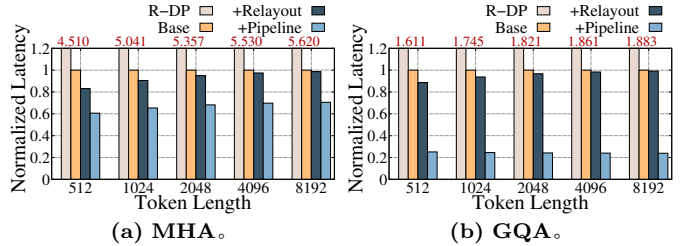


Fig. 14. 硬件消融研究。CHIME 朴素实现的延迟归一化为 1。

在各模型上的优势均有所减弱。原因在于，短上下文场景中增大批大小的边际收益递减。这一观察也与第 III-D 节的分析一致。

可扩展性分析。我们进一步使用不同 CHIME-PIM 配置评估 CHIME 性能，结果表明，性能可扩展性要求同时扩展内存容量和带宽。我们首先把“1 个秩组、512GB”设为基础内存配置。在此基础上，不同基线以不同方式扩展内存：“Bw-only”表示只通过增加秩组扩展内存带宽，而容量保持 512GB；“Cap-only”表示只把容量从 512GB 扩展至 4TB；“Bw-Cap”表示同时扩展带宽与容量。我们从轨迹中执行 1000 个请求并计算平均吞吐量。

图 13 展示 GPT-175B 与 QWEN-72B 在 OpenR1 轨迹上的结果。其他模型与轨迹的结果相似。结果表明，仅扩展带宽或容量均不能有效提高吞吐量。例如，在 GPT-175B 上单独将内存容量或带宽扩大 8 $\times$ 时，吞吐量分别只增加 2.28 $\times$ 和 1.01 $\times$ 。相反，同时扩大带宽与容量时，吞吐量增加 8.23 $\times$ 。这表明 CHIME 能够有效利用容量和带宽两方面的可扩展性价值。

C. 消融研究

CHIME-PIM 的消融研究。我们分析以下硬件优化：(1) 无气泡流水线（第 V-A 节）；(2) 混合粒度重布局（第 V-B 节）。我们实现了一个基线 Bank 级 CHIME-PIM，它将 Head 计算映射到多个芯片。按照既有工作 [68]，我们保守地把 CPU 侧重布局成本估算为：读取一种布局存储的数

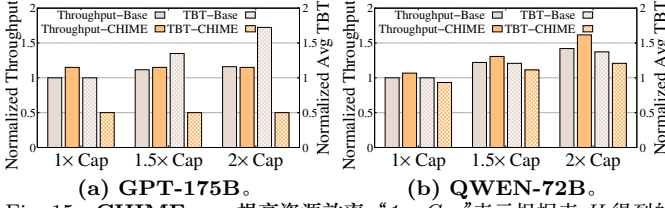


Fig. 15. **CHIME-sys** 提高资源效率。“1× Cap”表示根据表 II 得到的基础内存容量 (GPT-175B 为 2TB, QWEN-72B 为 1TB)。结果使用 *OpenR1* 轨迹评估。

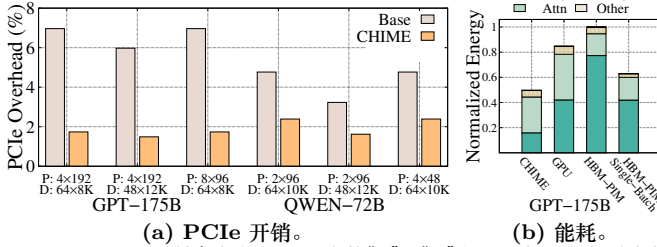


Fig. 16. **PCIe** 开销与能耗。(a) 中的“P”和“D”表示一个批次内预填充与解码请求的批大小和 *Token* 长度；(b) 中“GPU”的能耗归一化为 1。

据并按另一种布局写回所需的内存访问时间。不同 *Token* 长度下的性能结果见图 14。第一，无气泡流水线在 MHA 和 GQA 计算中分别将延迟降低约 27.9% 和 74.4%，这归功于重叠执行与特定 Head 映射方法。第二，硬件混合粒度重布局最高可将延迟降低 17%。随着 *Token* 长度增加，收益占比下降，是因为 Attention 计算逐渐主导总体执行时间。不过，由于每一层、每次生成 *Token* 都会累积重布局开销，我们认为该优化仍有必要。采用全部硬件优化后，CHIME 可实现 1.42–4.18× 的加速。此外，所有 Bank 级实现相对 Rank 级 DIMM-PIM (R-DP) 均可实现超过 1.5× 的加速。

调度器消融研究。我们评估 CHIME 调度器如何有效提高资源利用率。图 15 的左、右纵轴分别展示不同调度方法的平均吞吐量和 *Token* 间隔时间 (TBT)。基线表示优先填满 CHIME-PIM 容量的调度策略，CHIME 则表示对齐预测调度。结果表明，CHIME 调度器可以在不牺牲吞吐量的情况下，将延迟最高降低 70.93%，说明它能够消除 GPU 侧的空闲气泡。通过使包含预填充请求的批次对齐并改善负载均衡，CHIME 甚至能够略微提高吞吐量。此外，对 MHA 模型而言，如果增加 CHIME-PIM 容量，基线会选择更大的批大小并导致 TBT 升高，却不能提高吞吐量（因为 Attention 成为瓶颈）；而 CHIME 可以避免产生气泡，阻止 TBT 增长。CHIME 在 MHA 模型上的表现更好；原因在于，对 GQA 模型而言，要让 Attention 成为瓶颈，需要选择占用更大内存容量的请求，甚至可能根本无法达到 Attention 瓶颈。

PCIe 开销分析。我们评估采用与不采用秩组粒度通信—计算重叠时，跨设备 PCIe 通信如何降低吞吐量。如图 16 所示，在不同批次配置下，采用该优化可将开销最高降低 75.08%。这表明，DGX-A100 的 4 个秩组能够以秩组为粒度独立通信和计算，从而隐藏大部分 PCIe 开销。

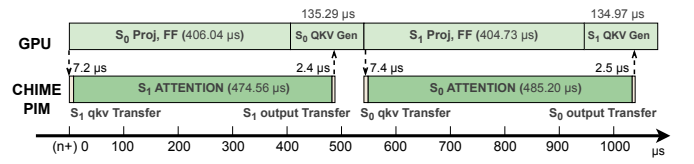


Fig. 17. 单个 LLM 层中 **CHIME** 流水线的解码执行时间线。该批次使用 GPT-175B 与 *OpenR1* 轨迹执行。

D. 成本分析

能耗。我们评估了 CHIME、GPU 和 HBM-PIM 的能耗。GPU 侧能耗采用 AttAcc 模拟器中的评估方法。对 PIM 侧能耗，第一步使用功耗参数（如 VDD、IDD）与时序参数（如 tRAS、tRP），计算 ACT、RD、WR 等操作的能耗 [55], [56]。第二步按照既有缩放方法 [59], [61], [62]，计算 DRAM 芯片（或 DRAM Die）内部读写（如 MAC、加载共享缓冲区）的能耗。第三步从 CACTI-IO [31] 获得芯片 I/O 能耗。我们提供三个基线：GPU、HBM-PIM，以及不采用子批次调度的 HBM-PIM。

图 16-b 详细展示 GPT-175B 在 *OpenR1* 轨迹上的能耗分解，其中 FC 与 Attention 计算占据主导。第一，CHIME 将总能耗降低约 40%，主要原因是在 GPU 上用更大的批大小执行 FC，减少了加载权重的次数。第二，采用子批次调度的 HBM-PIM 会消耗更多能量，因为它把批大小减半。第三，在 Attention 计算中，CHIME-PIM 的多芯片分布式计算能耗与 GPU 相当，但高于 HBM-PIM。

硬件开销。我们评估 Bank PU 的面积开销；它们集成在 DRAM 芯片中，会直接影响内存容量。假设命令解码和数据路径复用现有 DRAM 逻辑，我们使用 TSMC 28nm 工艺综合该设计。单个 Bank PU 和共享缓冲区分别占用 0.0032 mm² 和 0.051 mm²。考虑到逻辑工艺与 DRAM 工艺的密度差异，我们估计在 1z-nm DRAM 节点上实现时，综合面积约会翻倍。虽然高效支持 GQA 要求按比例扩展 Bank PU，但基于两个原因，我们认为该开销完全可以接受。第一，DIMM 的巨大容量能够有效摊薄面积成本。此外，新兴 3D 集成技术可以进一步缓解甚至消除这种 Die 面积代价。

时间线分解分析。如图 17 所示，我们从端到端评估的实际执行中提取一个具有代表性的批次作为示例，并进行详细的时间线分解分析。我们观察到，采用本文设计后，GPU 与 PIM 两侧的计算能够以较少气泡有效组成流水线。

VIII. 相关工作

用于 LLM 推理的 PIM/PNM 架构。除 AFD 系统中用于批量推理的 PIM 设计 [28], [61] 外，IANUS [50], [67] 采用不同的映射方法加速非批量推理。针对长上下文场景，LoL-PIM [40]、CXL-PNM [63] 和 HPU [66] 提出可扩展的容量与带宽；DRM 可以指导这些系统的配置以获得最优性能。Duplex [79] 面向混合专家 (MoE) 推理。由于 MoE 模型的 FC 层与稠密模型呈现相似的性能模式，DRM 可直接扩展用于分析 MoE 模型。Papi [27] 针对不同批大小，在多种 FC 设备上提出动态调度方法。我们认为该工作与本文正交；这些设备对应 DRM 模型中的不同 Line-FC，并在低每秒请求数等若干真实场景中展现出较高的性能潜力。CENT [22] 提出可扩展的纯 PIM 推理，可在多种场景

中实现高性能。但在 AFD 推理场景中，其 GDDR6-PIM 的成本效率会下降，如图 3 所示。

扩展 KV cache 容量的系统。FastDecode [25] 与 NEO [30] 把 KV cache 卸载到主机内存，并由 CPU 计算 Attention。InstAttention [60] 利用 SSD 内部带宽，通过计算存储盘卸载 Attention。如第 III-A 节所分析，这些系统受限于 Attention 计算带宽。CachedAttention [20]、FlexGen [69] 等工作使用主机内存存储 KV cache，再将其取回 GPU 进行 Attention 计算。其延迟受 PCIe 带宽限制。

仅使用 GPU 的 AFD 系统。部分既有工作在异构 GPU 上分离 Attention 与 FC [71], [72], [83]。基于 DRM 的方法可以帮助它们更有效地平衡 Attention 与 FC 计算。GPU 具有更强的通用性和软件兼容性，而 DIMM-PIM 则具有显著的成本优势。

IX. 结论

本文提出首个通用 AFD 性能模型——分离式屋顶线模型，并由此总结出指导 CHIME 设计的“利比希定律”。CHIME 是一个软硬件协同设计、采用 DIMM-PIM 的 AFD 系统，可为 LLM 推理提供可扩展的内存容量与带宽。CHIME 能够高效执行 DIMM-PIM Attention 计算，并最大化跨设备推理的资源利用率。评估结果表明，CHIME 可实现显著更高的吞吐量，为高效长上下文 LLM 推理建立了一种新范式。

致谢

我们感谢 ISCA 匿名审稿人提出的建设性意见。本工作部分得到国家自然科学基金（编号 62302300、62432010、62472279 和 U24B20165）、教育部基础学科和交叉学科突破计划（JYB2025XDXM122）、中央高校基本科研业务费以及面向智能体操作系统前沿研究中心（TC20260106010）的支持。通讯作者：Dong Du (dd_nirvana@sjtu.edu.cn) 和 Naifeng Jing (sjtuj@sjtu.edu.cn)。

参考文献

- [1] “Nvidia readying h20 ai gpu for chinese market,” <https://www.techpowerup.com/318595/nvidia-readying-h20-ai-gpu-for-chinese-market>, 2024.
- [2] “cognitivecomputations/dolphin-r1 · datasets at hugging face,” <https://huggingface.co/datasets/cognitivecomputations/dolphin-r1>, 2025, referenced April 2025.
- [3] “Keepa: search for ddr4 lrdimm,” <https://keepa.com/#!search/1-DDR42025>.
- [4] “Keepa: search for ddr5 rdimm,” <https://keepa.com/?&#!search/1-DDR5>.
- [5] “open-r1/openr1-math-220k · datasets at hugging face,” <https://huggingface.co/datasets/open-r1/OpenR1-Math-220k>, 2025, referenced April 2025.
- [6] “open-r1/openthoughts-114k-math · datasets at hugging face,” <https://huggingface.co/datasets/open-r1/OpenThoughts-114k-math>, 2025, referenced April 2025.
- [7] “Inside nvidia grace cpu: Nvidia amps up superchip engineering for hpc and ai,” <https://developer.nvidia.com/blog/inside-nvidia-grace-cpu-nvidia-amps-up-superchip-engineering-for-hpc-and-ai/>, 2026.
- [8] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Taming throughput-latency tradeoff in llm inference with sarathi-serve,” in *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI’24. USA: USENIX Association, 2024.

- [9] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, E. Chu, J. H. Clark, L. E. Shafey, Y. Huang, K. Meier-Hellstern, G. Mishra, E. Moreira, M. Omernick, K. Robinson, S. Ruder, Y. Tay, K. Xiao, Y. Xu, Y. Zhang, G. H. Abrego, J. Ahn, J. Austin, P. Barham, J. Botha, J. Bradbury, S. Brahma, K. Brooks, M. Catasta, Y. Cheng, C. Cherry, C. A. Choquette-Choo, A. Chowdhery, C. Crepy, S. Dave, M. Dehghani, S. Dev, J. Devlin, M. Díaz, N. Du, E. Dyer, V. Feinberg, F. Feng, V. Fienber, M. Freitag, X. Garcia, S. Gehrmann, L. Gonzalez, G. Gur-Ari, S. Hand, H. Hashemi, L. Hou, J. Howland, A. Hu, J. Hui, J. Hurwitz, M. Isard, A. Ittycheriah, M. Jagielski, W. Jia, K. Kenealy, M. Krikun, S. Kudugunta, C. Lan, K. Lee, B. Lee, E. Li, M. Li, W. Li, Y. Li, J. Li, H. Lim, H. Lin, Z. Liu, F. Liu, M. Maggioni, A. Mahendru, J. Maynez, V. Misra, M. Moussalem, Z. Nado, J. Nham, E. Ni, A. Nystrom, A. Parrish, M. Pellat, M. Polacek, A. Polozov, R. Pope, S. Qiao, E. Reif, B. Richter, P. Riley, A. C. Ros, A. Roy, B. Saeta, R. Samuel, R. Shelby, A. Slone, D. Smilkov, D. R. So, D. Sohn, S. Tokumine, D. Valter, V. Vasudevan, K. Vodrahalli, X. Wang, P. Wang, Z. Wang, T. Wang, J. Wieting, Y. Wu, K. Xu, Y. Xu, L. Xue, P. Yin, J. Yu, Q. Zhang, S. Zheng, C. Zheng, W. Zhou, D. Zhou, S. Petrov, and Y. Wu, “Palm 2 technical report,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.10403>
- [10] H. Asghari-Moghaddam, Y. H. Son, J. H. Ahn, and N. S. Kim, “Chameleon: versatile and practical near-dram acceleration architecture for large memory systems,” in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-49. IEEE Press, 2016.
- [11] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “Longbench: A bilingual, multitask benchmark for long context understanding,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.14508>
- [12] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, “Moe-lightning: High-throughput moe inference on memory-constrained gpus,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 715–730. [Online]. Available: <https://doi.org/10.1145/3669940.3707267>
- [13] L. Chen, D. Lyu, J. Jiang, Q. Wang, Z. Mao, and N. Jing, “Asyncdimm: Achieving asynchronous execution in dimm-based near-memory processing,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 518–532.
- [14] W. Cui, H. Zhao, Q. Chen, N. Zheng, J. Leng, J. Zhao, Z. Song, T. Ma, Y. Yang, C. Li, and M. Guo, “Enable simultaneous dnn services based on deterministic operator overlap and precise latency prediction,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3476143>
- [15] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: fast and memory-efficient exact attention with io-awareness,” in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS ’22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [16] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu,

- S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.12948>
- [17] C. Delimitrou and C. Kozyrakis, “Paragon: Qos-aware scheduling for heterogeneous datacenters,” *SIGPLAN Not.*, vol. 48, no. 4, p. 77–88, Mar. 2013. [Online]. Available: <https://doi.org/10.1145/2499368.2451125>
- [18] F. Devaux, “The true processing in memory accelerator,” in *2019 IEEE Hot Chips 31 Symposium (HCS)*, 2019, pp. 1–24.
- [19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [20] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, “Cost-Efficient large language model serving for multi-turn conversations with CachedAttention,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 111–126. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [21] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhennde, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelen, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A. L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Muxa, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, “The llama 3 herd of models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [22] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, “Pim is all you need: A cxl-enabled gpu-free system for large language model inference,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, Mar. 2025, pp. 862–881. [Online]. Available: <https://arxiv.org/abs/2407.21783>

<http://dx.doi.org/10.1145/3676641.3716267>

- [23] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like clockwork: Performance predictability from the bottom up," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. USENIX Association, Nov. 2020, pp. 443–462. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/gujarati>
- [24] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52 565–52 608, 2022.
- [25] J. He and J. Zhai, "Fastdecode: High-throughput gpu-efficient llm serving using heterogeneous pipelines," 2024. [Online]. Available: <https://arxiv.org/abs/2403.11421>
- [26] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 372–385.
- [27] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "Papi: Exploiting dynamic parallelism in large language model decoding with a processing-in-memory-enabled computing system," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 766–782. [Online]. Available: <https://doi.org/10.1145/3676641.3716009>
- [28] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 722–737. [Online]. Available: <https://doi.org/10.1145/3620666.3651380>
- [29] J. Huang and K. C.-C. Chang, "Towards reasoning in large language models: A survey," 2023. [Online]. Available: <https://arxiv.org/abs/2212.10403>
- [30] X. Jiang, Y. Zhou, S. Cao, I. Stoica, and M. Yu, "Neo: Saving gpu memory crisis with cpu offloading for online llm inference," 2024. [Online]. Available: <https://arxiv.org/abs/2411.01142>
- [31] N. P. Jouppi, A. B. Kahng, N. Muralimanohar, and V. Srinivas, "Cacti-io: Cacti with off-chip power-area-timing models," in *2012 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2012, pp. 294–301.
- [32] A. K. Kakolyris, D. Masouros, P. Vavaroutsos, S. Xydis, and D. Soudris, "throttl'em: Predictive gpu throttling for energy efficient llm inference serving," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1363–1378.
- [33] K. S. Kalyan, "A survey of gpt-3 family large language models including chatgpt and gpt-4," 2023. [Online]. Available: <https://arxiv.org/abs/2310.12321>
- [34] L. Ke, U. Gupta, B. Y. Cho, D. Brooks, V. Chandra, U. Diril, A. Firoozshahian, K. Hazelwood, B. Jia, H.-H. S. Lee, M. Li, B. Maher, D. Mudigere, M. Naumov, M. Schatz, M. Smelyanskiy, X. Wang, B. Reagen, C.-J. Wu, M. Hempstead, and X. Zhang, "Recnmp: Accelerating personalized recommendation with near-memory processing," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 790–803.
- [35] L. Ke, X. Zhang, J. So, J.-G. Lee, S.-H. Kang, S. Lee, S. Han, Y. Cho, J. H. Kim, Y. Kwon, K. Kim, J. Jung, I. Yun, S. J. Park, H. Park, J. Song, J. Cho, K. Sohn, N. S. Kim, and H.-H. S. Lee, "Near-memory processing in action: Accelerating personalized recommendation with axdimm," *IEEE Micro*, vol. 42, no. 1, pp. 116–127, 2022.
- [36] G. Kim, J. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun, J. Kim, C. Kim, I. Kim, J. Park, Y. Song, B. Yang, H. Lee, S. Park, W. Lee, S. Kim, Y. Park, Y. Jung, G.-H. Park, and E. Lim, "Sk hynix ai-specific computing memory solution: From aim device to heterogeneous aimx-xpu system for comprehensive llm inference," in *2024 IEEE Hot Chips 36 Symposium (HCS)*, 2024, pp. 1–26.
- [37] H. Kim, H. Park, T. Kim, K. Cho, E. Lee, S. Ryu, H.-J. Lee, K. Choi, and J. Lee, "Gradpim: A practical processing-in-dram architecture for gradient descent," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 249–262.
- [38] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah, J. Choi, J. So, Y. Cho, J. Song, J. Choi, J. Cho, K. Sohn, Y. Sohn, K. Park, and N. S. Kim, "Aquabolt-xl: Samsung hbm2-pim with in-memory processing for ml accelerators and beyond," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, 2021, pp. 1–26.
- [39] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah, J. Choi, J. So, Y. Cho, J. Song, J. Choi, J. Cho, K. Sohn, Y. Sohn, K. Park, and N. S. Kim, "Aquabolt-xl: Samsung hbm2-pim with in-memory processing for ml accelerators and beyond," in *2021 IEEE Hot Chips 33 Symposium (HCS)*, 2021, pp. 1–26.
- [40] H. Kwon, K. Koo, J. Kim, W. Lee, M. Lee, H. Lee, Y. Jung, J. Park, Y. Song, B. Yang, H. Choi, G. Kim, J. Won, W. Shin, C. Kim, G. Shin, Y. Kwon, I. Kim, E. Lim, J. Kim, and J. Choi, "Lol-pim: Long-context llm decoding with scalable dram-pim system," 2025. [Online]. Available: <https://arxiv.org/abs/2412.20166>
- [41] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [42] Y. Kwon, G. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun, J. Kim, C. Kim, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Park, W. Lee, S. Lee, K. Kim, D. Kwon, C. Jeong, J. Kim, E. Lim, and J. Chun, "Memory-centric computing with sk hynix's domain-specific memory," in *2023 IEEE Hot Chips 35 Symposium (HCS)*, 2023, pp. 1–26.
- [43] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, J. Chun, and S. Hynix, "System architecture and software stack for gddr6-aim," in *2022 IEEE Hot Chips 34 Symposium (HCS)*, 2022, pp. 1–25.
- [44] D. Lee, J. So, M. AHN, J.-G. Lee, J. Kim, J. Cho, R. Oliver, V. C. Thummala, R. s. JV, S. S. Upadhyay, M. I. Khan, and J. H. Kim, "Improving in-memory database operations with acceleration dimm (axdimm)," in *Proceedings of the 18th International Workshop on Data Management on New Hardware*, ser. DaMoN '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3533737.3535093>
- [45] D. Lee, B. Hyun, T. Kim, and M. Rhu, "Pim-mm: A memory management unit for accelerating data transfers in commercial pim systems," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 627–642.
- [46] H. Lee, D. Baek, J. Son, J. Choi, K. Moon, and M. Jang, "Paise: Pim-accelerated inference scheduling engine for transformer-based llm," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1707–1719.
- [47] H. Lee, G. Kim, D. Yun, I. Kim, Y. Kwon, and E. Lim, "Cost-effective llm accelerator using processing in memory technology," in *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*, 2024, pp. 1–2.
- [48] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, "A lynn 1.25v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting

- 1tflops mac operation and various activation functions for deep-learning applications,” in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.
- [49] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware architecture and software stack for pim based on commercial dram technology : Industrial product,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.
- [50] C. Li, Y. Yin, X. Wu, J. Zhu, Z. Gao, D. Niu, Q. Wu, X. Si, Y. Xie, C. Zhang, and G. Sun, “H2-llm: Hardware-dataflow co-exploration for heterogeneous hybrid-bonding-based low-batch llm inference,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 194–210. [Online]. Available: <https://doi.org/10.1145/3695053.3731008>
- [51] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “Dramsim3: A cycle-accurate, thermal-capable dram simulator,” *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 106–109, Jul. 2020. [Online]. Available: <https://doi.org/10.1109/LCA.2020.2973991>
- [52] L. Liu, S. Zhao, B. Li, H. Ren, Z. Xu, M. Wang, X. Li, Y. Han, and Y. Wang, “Make llm inference affordable to everyone: Augmenting gpu memory with ndp-dimm,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.16963>
- [53] Q. Liu, Y. Yang, D. Du, Y. Xia, P. Zhang, J. Feng, J. R. Larus, and H. Chen, “Harmonizing efficiency and practicability: Optimizing resource utilization in serverless computing with jiagu,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 1–17. [Online]. Available: <https://www.usenix.org/conference/atc24/presentation/liu-qingyuan>
- [54] D. Lo, L. Cheng, R. Govindaraju, P. Ranganathan, and C. Kozyrakis, “Heracles: improving resource efficiency at scale,” vol. 43, no. 3S. New York, NY, USA: Association for Computing Machinery, Jun. 2015, p. 450–462. [Online]. Available: <https://doi.org/10.1145/2872887.2749475>
- [55] “DRAM power calculator,” [Online], Micron Technology, 2017, available: <https://www.micron.com/sales-support/design-tools/dram-power-calculator?srsltid=AfmBOoq5fElzP8dXZW59LhcNk6lRj8mJb2x0MIPBcc8fXQlzia8X70uJ>
- [56] “Technical Note Calculating Memory Power for DDR4 SDRAM Introduction,” [Online], Micron Technology, 2017, available: https://www.mouser.com/pdfDocs/tn4007_ddr4_power_calculation.pdf?srsltid=AfmBOor9Axc_mw_rsqGmkNM8EWV-kw7xYUOnTICSjstofcX5TH2b6vJB
- [57] “NVIDIA DGX A100,” [Online], NVIDIA, 2023, available: <https://resources.nvidia.com/enus-dgx-systems/dgx-ai>
- [58] “NVIDIA Blackwell. The engine of the new industrial revolution,” [Online], NVIDIA, 2026, available: <https://www.primeline-solutions.com/media/categories/server/nach-gpu/nvidia-hgx-h200/nvidia-blackwell-b200-datasheet.pdf>
- [59] M. O’Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, “Fine-grained dram: Energy-efficient dram for extreme bandwidth systems,” in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.
- [60] X. Pan, E. Li, Q. Li, S. Liang, Y. Shan, K. Zhou, Y. Luo, X. Wang, and J. Zhang, “Instattention: In-storage attention offloading for cost-effective long-context llm inference,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, March 2025, pp. 1510–1525.
- [61] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “Attacc! unleashing the power of pim for batched transformer-based generative model inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 103–119. [Online]. Available: <https://doi.org/10.1145/3620665.3640422>
- [62] J. Park, B. Kim, S. Yun, E. Lee, M. Rhu, and J. H. Ahn, “Trim: Enhancing processor-memory interfaces with scalable tensor reduction in memory,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 268–281. [Online]. Available: <https://doi.org/10.1145/3466752.3480080>
- [63] S.-S. Park, K. Kim, J. So, J. Jung, J. Lee, K. Woo, N. Kim, Y. Lee, H. Kim, Y. Kwon, J. Kim, J. Lee, Y. Cho, Y. Tai, J. Cho, H. Song, J. H. Ahn, and N. S. Kim, “An lpddr-based cxl-pnm platform for tco-efficient inference of transformer-based large language models,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 970–982.
- [64] “PCI Express Specifications,” [Online], PCI-SIG, 2025, available: <https://pcisig.com/specifications>
- [65] A. Plaat, A. Wong, S. Verberne, J. Broekens, N. van Stein, and T. Back, “Reasoning with large language models, a survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.11511>
- [66] M. Rhee, J. Sim, T. Ahn, S. Lee, D. Yoon, E. Kim, K. Park, Y. Joo, and H. Kim, “Hpu: High-bandwidth processing unit for scalable, cost-effective llm inference via gpu co-processing,” *arXiv preprint arXiv:2504.16112*, 2025.
- [67] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin, J. Won, H. Choi, K. Kim, D. Kwon, C. Jeong, S. Lee, Y. Choi, W. Byun, S. Baek, H.-J. Lee, and J. Kim, “Ianus: Integrated accelerator based on npu-pim unified memory system,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 545–560. [Online]. Available: <https://doi.org/10.1145/3620666.3651324>
- [68] S. H. Seo, J. Kim, D. Lee, S. Yoo, S. Moon, Y. Park, and J. W. Lee, “Facil: Flexible dram address mapping for soc-pim cooperative on-device llm inference,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1720–1733.
- [69] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: high-throughput generative inference of large language models with a single gpu,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML’23. JMLR.org, 2023.
- [70] J. Stojkovic, C. Zhang, I. Goiri, J. Torrellas, and E. Choukse, “Dynamollm: Designing llm inference clusters for performance and energy efficiency,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1348–1362.
- [71] S. Team, “Step3: Cost-effective multimodal intelligence,” [Online]. Available: <https://stepfun.ai/research/step3>
- [72] S. Team, “Step-3 is large yet affordable: Model-system co-design for cost-effective decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.19427>
- [73] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, pp. 6000–6010.
- [74] Y. Wei, Y. C. Huang, H. Tang, N. Sankaran, I. Chadha, D. Dai, O. Oluwale, V. Balan, and E. Lee, “9.3 nvlink-c2c: A coherent off package chip-to-chip interconnect with 40gbps/pin single-ended signaling,” in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, 2023, pp. 160–162.
- [75] Wikipedia, “Liebig’s law of the minimum,” 2025. [Online]. Available: https://en.wikipedia.org/wiki/Liebig%27s_law_of_the_minimum
- [76] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, p. 65–76, Apr. 2009. [Online]. Available: <https://doi.org/10.1145/1498765.1498785>
- [77] Y. Yang, L. Zhao, Y. Li, H. Zhang, J. Li, M. Zhao, X. Chen, and K. Li, “Influss: a native serverless system for low-latency, high-throughput inference,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’22. New York, NY, USA: Association for

- Computing Machinery, 2022, p. 768–781. [Online]. Available: <https://doi.org/10.1145/3503222.3507709>
- [78] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/yu>
- [79] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, “Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1429–1443.
- [80] L. Zhao, Y. Yang, Y. Li, X. Zhou, and K. Li, “Understanding, predicting and scheduling serverless workloads under partial interference,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3476215>
- [81] T. Zhong, Z. Liu, Y. Pan, Y. Zhang, Y. Zhou, S. Liang, Z. Wu, Y. Lyu, P. Shu, X. Yu, C. Cao, H. Jiang, H. Chen, Y. Li, J. Chen, H. Hu, Y. Liu, H. Zhao, S. Xu, H. Dai, L. Zhao, R. Zhang, W. Zhao, Z. Yang, J. Chen, P. Wang, W. Ruan, H. Wang, H. Zhao, J. Zhang, Y. Ren, S. Qin, T. Chen, J. Li, A. H. Zidan, A. Jahin, M. Chen, S. Xia, J. Holmes, Y. Zhuang, J. Wang, B. Xu, W. Xia, J. Yu, K. Tang, Y. Yang, B. Sun, T. Yang, G. Lu, X. Wang, L. Chai, H. Li, J. Lu, L. Sun, X. Zhang, B. Ge, X. Hu, L. Zhang, H. Zhou, L. Zhang, S. Zhang, N. Liu, B. Jiang, L. Kong, Z. Xiang, Y. Ren, J. Liu, X. Jiang, Y. Bao, W. Zhang, X. Li, G. Li, W. Liu, D. Shen, A. Sikora, X. Zhai, D. Zhu, and T. Liu, “Evaluation of openai o1: Opportunities and challenges of agi,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.18486>
- [82] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li, S. Yan, G. Dai, X.-P. Zhang, Y. Dong, and Y. Wang, “A survey on efficient inference for large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14294>
- [83] R. Zhu, Z. Jiang, C. Jin, P. Wu, C. A. Stuardo, D. Wang, X. Zhang, H. Zhou, H. Wei, Y. Cheng, J. Xiao, X. Zhang, L. Liu, H. Lin, L.-W. Chang, J. Ye, X. Yu, X. Liu, X. Jin, and X. Liu, “Megascall-infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism,” in *Proceedings of the ACM SIGCOMM 2025 Conference*, ser. SIGCOMM ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 592–608. [Online]. Available: <https://doi.org/10.1145/3718958.3750506>